

Namal University Mianwali

Department of Computer Science

CSC-340: Theory of Automata | Semester Project

INITIAL PROJECT REPORT

Automata Simulator

An Interactive Cross-Platform Tool for Simulating
Finite Automata, Transition Graphs, Pushdown Automata,
Context-Free Grammars, and Turing Machines

COURSE	CSC-340: Theory of Automata
INSTRUCTOR	Sir Muhammad Bilal
SUBMITTED BY	Mehroz Ali Khan
PARTNER	Abdul Rehman
TECHNOLOGY	Flutter (Dart) — Web & Android
DEPLOYMENT	GitHub Pages (Free)
DATE	June 2, 2026

Contents

1	Introduction	2
2	Problem Statement and Motivation	2
3	Alignment with Course Syllabus (CSC-340)	3
4	Project Objectives	3
4.1	Primary Objectives	3
4.2	Secondary Objectives	4
5	Technology Stack and Free Deployment	4
5.1	Why Flutter?	5
5.2	Zero-Cost Deployment Stack	5
5.3	Technology Stack Summary	6
6	Feature Specification	6
6.1	Module 1: Deterministic Finite Automata (DFA)	6
6.2	Module 2: Non-Deterministic Finite Automata (NFA) and Transition Graphs	7
6.3	Module 3: Moore and Mealy Machines	7
6.4	Module 4: Pushdown Automata (PDA)	7
6.5	Module 5: Context-Free Grammars (CFG)	8
6.6	Module 6: Turing Machine (TM)	8
7	Sample Automata Diagrams	8
7.1	Example: DFA accepting strings ending in “ab”	9
7.2	Example: PDA for $\{a^n b^n \mid n \geq 1\}$	9
8	References	9

1 Introduction

Project Overview

The **Automata Simulator** is an interactive, visual, cross-platform educational tool developed as the semester project for **CSC-340: Theory of Automata** at Namal University Mianwali. Built entirely using **Flutter and Dart**, it compiles to a web application and Android APK from a single free codebase, deployed at zero cost using GitHub Pages.

Theory of Automata is among the most foundational courses in Computer Science. Its concepts — from finite automata and regular expressions to context-free grammars and Turing machines — underlie compilers, interpreters, text processing, network protocols, and artificial intelligence. However, these are inherently abstract mathematical objects that are difficult to visualize from textbook descriptions alone.

This project bridges that gap. Rather than reading about a DFA as a 5-tuple, students will *draw* it on a canvas, *feed* it an input string, and *watch* the machine execute step by step — with active states highlighted, transitions animated, and acceptance or rejection clearly shown.

Note: This is an **Initial Report** submitted at the beginning of the project. It documents our goals, planned features, technical choices, and timeline. A Final Report will be submitted upon completion with implementation details, test cases, and results.

2 Problem Statement and Motivation

Students in CSC-340 encounter significant difficulty in developing *operational intuition* for automata. The textbook (Cohen, 2nd Ed.) provides formal definitions and diagrams, and JFLAP is suggested as a reference tool — however:

- **JFLAP** is a Java desktop application: not mobile-accessible, and increasingly outdated in its UI.
- Most online simulators support only DFA/NFA and miss **Transition Graphs (TG)**, **GTG**, **Moore/Mealy machines**, **CFG derivation trees**, and **Turing Machines**.
- None provide a seamless **step-by-step execution** with visual highlighting aligned with Cohen's notation.
- No existing free tool covers the **complete syllabus** of CSC-340 in one unified interface.

Our simulator is designed specifically around the CSC-340 course outline, covering *every* major computational model taught in this course.

3 Alignment with Course Syllabus (CSC-340)

The following table maps each unit of CSC-340 to the corresponding simulator module we will build:

Unit	Topic	Simulator Feature	Status
1	Theory of Computation	Glossary module: alphabets, strings, Kleene closure	Planned
2	Finite State Machines	DFA, NFA, TG, GTG, Moore, Mealy simulators	Planned
3	Regular Expressions	RE input, NFA construction, Kleene's theorem demo	Planned
4	Push Down Automata	PDA with animated stack visualization	Planned
5	Context-Free Grammars	CFG editor, parse trees, derivation, ambiguity check	Planned
6	Turing Machines	TM tape simulator with head movement animation	Planned

Key Point: Our simulator is the *only* known free tool that covers all six CSC-340 units in one unified cross-platform application aligned with Cohen's textbook notation.

4 Project Objectives

4.1 Primary Objectives

- DFA Simulator** — Visual construction and step-by-step simulation of Deterministic Finite Automata using transition tables and diagrams as taught in lectures 9–10.
- NFA Simulator** — Simulation of Non-Deterministic Finite Automata with ϵ -closure, parallel state sets, and NFA→DFA conversion via subset construction (lecture 11).
- Transition Graph (TG) Simulator** — Support for generalized transition

graphs as covered in lectures 12, 14 (GTG).

4. **Moore & Mealy Machine Simulator** — FA with output: Moore and Mealy machines (lectures 19–20).
5. **PDA Simulator** — Pushdown Automata with animated stack, CFG-to-PDA conversion support (lectures 29–30).
6. **Context-Free Grammar Tool** — CFG editor, derivation trees, parse trees, ambiguity detection (lectures 21–28).
7. **Turing Machine Simulator** — TM tape with animated read/write head (lectures 31–32).
8. **String Tester** — Test any input string on any machine with Accept/Reject output and full execution trace.
9. **Step-by-Step Execution** — Advance through computation one symbol at a time with state highlighting.

4.2 Secondary Objectives

1. Regular Expression to NFA conversion demonstrating Kleene's Theorem (lectures 13–15).
2. DFA Minimization using the Partitioning Method and Myhill-Nerode Theorem (lectures 17–18).
3. Pumping Lemma visualizer for regular languages (lecture 7).
4. Save and Load automata as JSON files.
5. Dark and Light theme toggle.
6. Completely free deployment: Flutter Web on GitHub Pages.

5 Technology Stack and Free Deployment

5.1 Why Flutter?

Reason	Details
Single Codebase	One Dart codebase compiles to Web, Android, and iOS
100% Free	Flutter SDK is open-source; no licenses required
No Backend Needed	All logic runs client-side; no server costs
Rich Canvas API	<code>CustomPainter</code> enables complex automata graph drawing
Future-Proof	Android APK can be generated from the same code later
Cohen-aligned	Can replicate the exact notation used in the course textbook

5.2 Zero-Cost Deployment Stack

Service	Cost	Purpose
Flutter SDK	Free (open-source)	Application framework and compiler
Dart Language	Free (open-source)	Programming language
GitHub	Free	Version control and code hosting
GitHub Pages	Free	Web deployment (<code>flutter build web</code>)
GitHub Actions	Free	Auto-build and deploy on push
Android APK	Free	Direct APK file shared via GitHub Releases

Deployment Workflow — Fully Automated

1. Code is pushed to the `main` branch on GitHub
2. GitHub Actions automatically runs `flutter build web`
3. Built files are deployed to GitHub Pages (`gh-pages` branch)
4. Live URL: <https://mehrozalikhan.github.io/automata-simulator/>
5. Android APK is attached to each GitHub Release

Monthly cost: PKR 0

5.3 Technology Stack Summary

Language	Dart 3.x
Framework	Flutter 3.x
State Mgmt	Provider / Riverpod (free, open-source)
Canvas	Flutter CustomPainter + GestureDetector
Persistence	shared_preferences + JSON (no database needed)
Web Build	flutter build web → GitHub Pages
Android Build	flutter build apk → GitHub Releases
CI/CD	GitHub Actions (free tier)
Version Control	Git + GitHub (free)

6 Feature Specification

6.1 Module 1: Deterministic Finite Automata (DFA)

Feature	Priority	Description
Add / Delete States	Must Have	Click canvas to place named state node
Drag States	Must Have	Freely reposition on canvas
Set Start State	Must Have	Entry arrow rendered automatically
Set Accept State	Must Have	Double circle notation (Cohen style)
Add Transition	Must Have	Draw arc; label with alphabet symbol
Self-Loop	Must Have	Transition from state to itself
Test String	Must Have	Accept / Reject output with trace
Step-by-Step	Must Have	One symbol at a time; highlight active state
Transition Table View	Must Have	Toggle between diagram and table view
DFA Minimization	Should Have	Partitioning method and Myhill-Nerode
Export as JSON	Should Have	Save and reload automaton

6.2 Module 2: Non-Deterministic Finite Automata (NFA) and Transition Graphs

Feature	Priority	Description
NFA Construction	Must Have	Multiple transitions on same symbol; ε -moves
ε-Closure	Must Have	Computed automatically and shown
Parallel Execution	Must Have	All active states highlighted simultaneously
NFA to DFA Conversion	Must Have	Subset construction algorithm
Transition Graph (TG)	Must Have	TG as defined in Cohen Ch.6
Generalized TG (GTG)	Should Have	RE-labeled transitions
Kleene's Theorem Demo	Should Have	Visual proof steps

6.3 Module 3: Moore and Mealy Machines

Feature	Priority	Description
Moore Machine Canvas	Must Have	States with output labels
Mealy Machine Canvas	Must Have	Transitions with input/output labels
String Execution	Must Have	Show output string produced
Moore = Mealy Demo	Should Have	Convert between the two representations

6.4 Module 4: Pushdown Automata (PDA)

Feature	Priority	Description
PDA Construction	Must Have	7-tuple: states, input/stack alphabets, transitions
Animated Stack	Must Have	Push/pop animated in real time during execution
String Testing	Must Have	Accept by final state or empty stack
CFG to PDA Conversion	Should Have	Automated construction from a given CFG

6.5 Module 5: Context-Free Grammars (CFG)

Feature	Priority	Description
CFG Editor	Must Have	Enter productions in standard notation
Derivation Viewer	Must Have	Leftmost/rightmost derivation step-by-step
Parse Tree Display	Must Have	Visual tree matching Cohen's derivation trees
Ambiguity Detection	Should Have	Show two distinct parse trees if grammar is ambiguous
CNF Conversion	Should Have	Chomsky Normal Form transformation
GNF Conversion	Should Have	Greibach Normal Form
CFL Simplification	Should Have	Remove useless symbols, unit productions, epsilons
Pumping Lemma (CFL)	Could Have	Visualizer for Pumping Lemma for CFLs

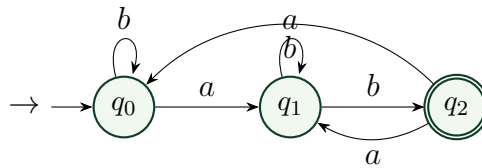
6.6 Module 6: Turing Machine (TM)

Feature	Priority	Description
TM Canvas	Must Have	State diagram construction
Infinite Tape View	Must Have	Scrollable tape with cell-by-cell display
Animated Read/Write Head	Must Have	Head moves left/right with symbol read/write shown
Step Execution	Must Have	One transition at a time
Accept/Reject/Loop	Must Have	Detect halt states and infinite loops (with limit)

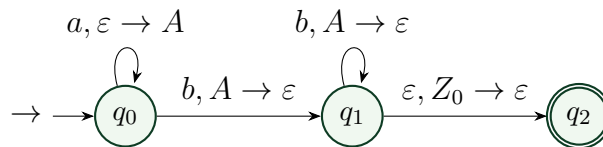
7 Sample Automata Diagrams

To illustrate the kind of visual output our simulator will produce, we provide two example diagrams drawn using TikZ (matching Cohen's notation style).

7.1 Example: DFA accepting strings ending in “ab”



7.2 Example: PDA for $\{a^n b^n \mid n \geq 1\}$



These are examples of what students will be able to construct interactively on the canvas.

8 References

1. Cohen, D. I. A. (1997). *Introduction to Computer Theory* (2nd ed.). John Wiley & Sons. [Primary textbook for CSC-]
2. Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage.
3. Bole, J. (2021). *Formal Languages and Automata Theory*. Independently published.
4. JFLAP Tutorial. <https://www.jflap.org/tutorial/>
5. Flutter Documentation (2024). <https://flutter.dev/docs>
6. Dart Language Guide (2024). <https://dart.dev/guides>
7. GitHub Pages Documentation. <https://pages.github.com>
8. GitHub Actions Documentation. <https://docs.github.com/en/actions>